WEJ LAB

Cross Validation

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-10-29

RUTGERS HEALTH
Center for Data Science

WEJ LAB

Learning objectives

By the end of this lecture you should be able to:

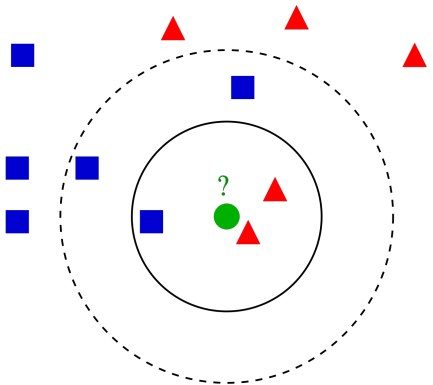
- Explain over-training and over-smoothing using kNN
- Describe K-fold cross-validation and the bootstrap
- Use cross-validation to choose a tuning parameter
- Fit and tune models with caret's `train` function
- Use CV to develop a TB gene-expression biomarker, and avoid selection bias

Cross-validation

Cross-validation is one of the most important ideas in machine learning. We start with the concepts and math, then implement it with `caret`.

K-nearest neighbors (kNN)

To motivate cross-validation, we start with a real algorithm: **k-nearest neighbors (kNN)**.



K-nearest neighbors (kNN)

K-nearest neighbor algorithm pseudocode

Programming languages like Python and R are used to implement the KNN algorithm. The following is the pseudocode for KNN:

1. Load the data
2. Choose K value
3. For each data point in the data:
    - Find the Euclidean distance to all training data samples
    - Store the distances on an ordered list and sort it
    - Choose the top K entries from the sorted list
    - Label the test point based on the majority of classes present in the selected points
4. End

K-nearest neighbors (kNN)

kNN estimates the conditional probability

$$p(x_1, x_2) = \Pr(Y = 1 \mid X_1 = x_1, X_2 = x_2).$$

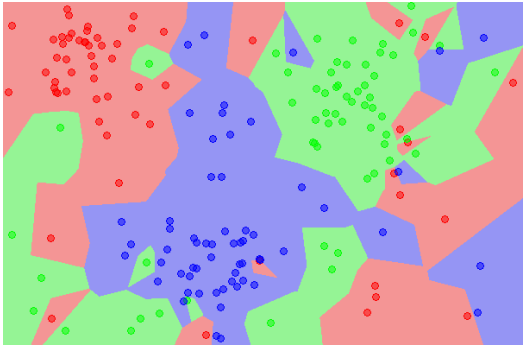
For any point (x_1, x_2):

1. Find the k nearest training points (the *neighborhood*)
2. Average their 0s and 1s to get $\hat{p}(x_1, x_2)$

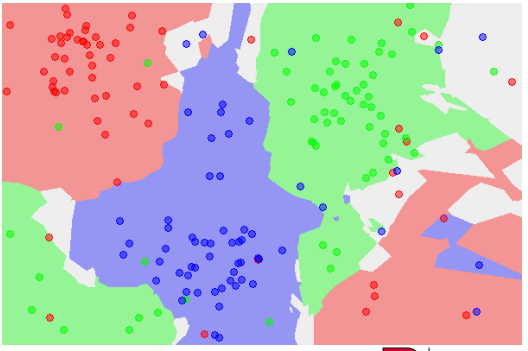
WEJ LAB
RUTGERS HEALTH
Center for Data Science

K-nearest neighbors (kNN)

Using k=*1*:



and k=5:



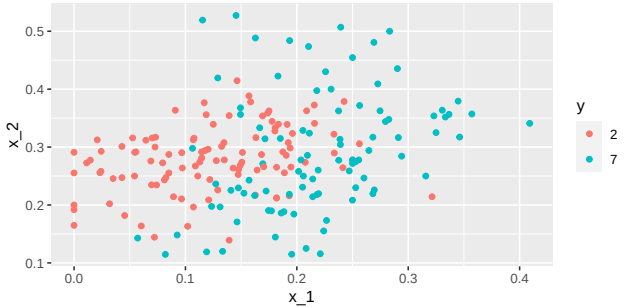
WEJ LAB

RUTGERS HEALTH
Center for Data Science

K-nearest neighbors (kNN): Example

Using the following dataset:

```
library(dslabs); data("mnist_27")
mnist_27$test %>% ggplot(aes(x_1, x_2, color = y)) + geom_point()
```



K-nearest neighbors (kNN): Example

Fit kNN with $k = 5$ using `caret::knn3`:

```
library(caret)
knn_fit <- knn3(y ~ ., data = mnist_27$train, k = 5)
```

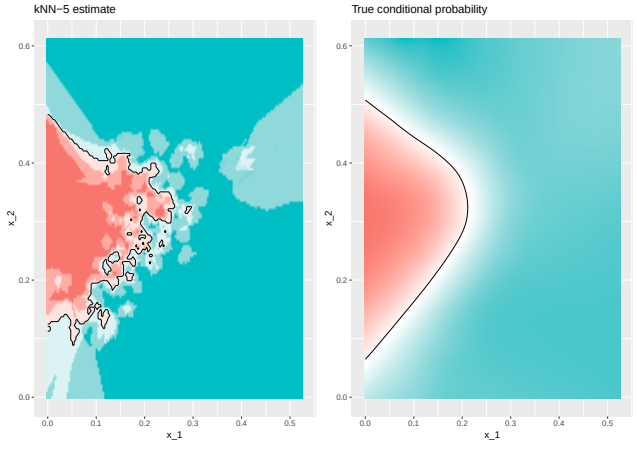
K-nearest neighbors (kNN): Example

The classes are balanced, so we use accuracy to measure performance. `type = "class"` returns predicted labels:

```
y_hat_knn <- predict(knn_fit, mnist_27$test, type = "class")
confusionMatrix(y_hat_knn, mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##    0.815
```

K-nearest neighbors (kNN): Example



Over-training

Over-training (over-fitting): accuracy is higher on the training set than on the test set:

```
y_hat_knn <- predict(knn_fit, mnist_27$train, type = "class")
confusionMatrix(y_hat_knn, mnist_27$train$y)$overall["Accuracy"]
```

```
## Accuracy
##   0.8825
```

```
y_hat_knn <- predict(knn_fit, mnist_27$test, type = "class")
confusionMatrix(y_hat_knn, mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##    0.815
```

Over-training

Over-training is worst with $k = 1$:

```
knn_fit_1 <- knn3(y ~ ., data = mnist_27$train, k = 1)
y_hat_knn_1 <- predict(knn_fit_1, mnist_27$train, type = "class")
confusionMatrix(y_hat_knn_1, mnist_27$train$y)$overall[["Accuracy"]]
```

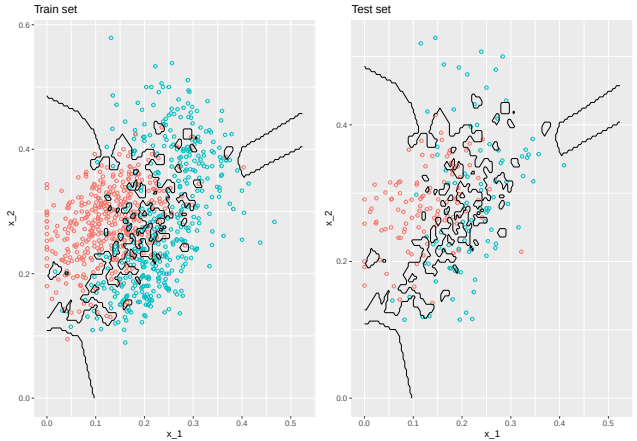
```
## [1] 0.9975
```

```
y_hat_knn_1 <- predict(knn_fit_1, mnist_27$test, type = "class")
confusionMatrix(y_hat_knn_1, mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##    0.735
```

Over-training

With $k = 1$, the boundary traces every training point:



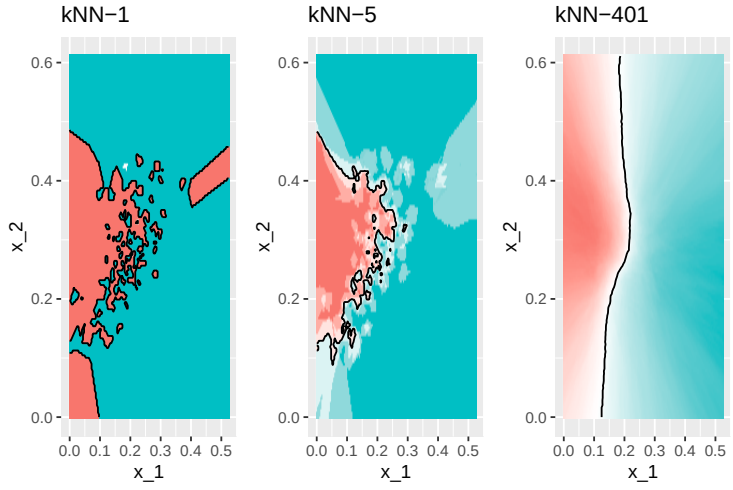
Over-smoothing

$k = 5$ is also over-trained, though less than $k = 1$. What about a much larger k, such as $k = 401$?

```
knn_fit_401 <- knn3(y ~ ., data = mnist_27$train, k = 401)
y_hat_knn_401 <- predict(knn_fit_401, mnist_27$test,
                         type = "class")
confusionMatrix(y_hat_knn_401, mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##     0.79
```

Over-smoothing



Picking the k in kNN

We want the k that maximizes accuracy (or minimizes expected MSE). **Cross validation** estimates these quantities for any algorithm and tuning parameter.

To see why we need it, try many values of k:

```
ks <- seq(3, 251, 2)
```

Picking the k in kNN

Use `map_df` to compute train and test accuracy for each k:

```
library(purrr)
accuracy <- map_df(ks, function(k){
  fit <- knn3(y ~ ., data = mnist_27$train, k = k)

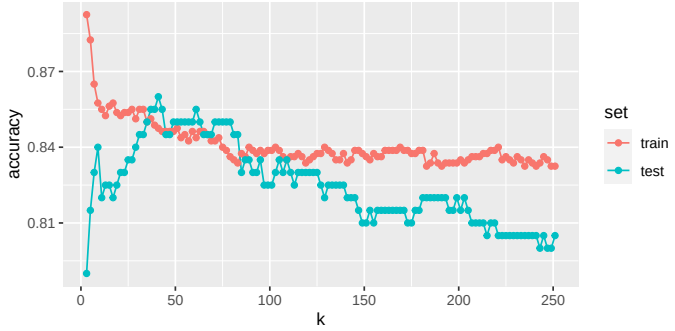
  y_hat <- predict(fit, mnist_27$train, type = "class")
  cm_train <- confusionMatrix(y_hat, mnist_27$train$y)
  train_error <- cm_train$overall["Accuracy"]

  y_hat <- predict(fit, mnist_27$test, type = "class")
  cm_test <- confusionMatrix(y_hat, mnist_27$test$y)
  test_error <- cm_test$overall["Accuracy"]

  tibble(train = train_error, test = test_error)
})
```

Picking the k in kNN

Accuracy for each k:



Picking the k in kNN

Picking k this way uses the test set, so this accuracy is over-optimistic:

```
ks[which.max(accuracy$test)]
```

```
## [1] 41
```

```
max(accuracy$test)
```

```
## [1] 0.86
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Mathematical description of cross validation

Goal: find predictions $\hat{Y}$ for outcome Y that minimize the MSE:

$$\text{MSE} = \text{E}\left\{\frac{1}{N}\sum_{i=1}^{N}(\hat{Y}_i - Y_i)^2\right\}$$

Mathematical description of cross validation

With only one dataset, we estimate it with the observed MSE:

$$\hat{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

These are the *true error* and the *apparent error*.

Mathematical description of cross validation

Two problems with the apparent error:

1. It is a random variable: one algorithm can beat another by luck.
2. If we train and evaluate on the same data, we overtrain (as with $k = 1$).

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Mathematical description of cross validation

Cross validation addresses both. Think of the true error as the average apparent error over B new random samples, none used for training:

$$\frac{1}{B}\sum_{b=1}^{B}\frac{1}{N}\sum_{i=1}^{N}\left(\hat{y}_i^b - y_i^b\right)^2$$

with B practically infinite.

Mathematical description of cross validation

In practice we only have one set of outcomes, $y_1, \ldots, y_n$. Cross validation imitates this setup by generating many random samples from the data we have.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

K-fold cross validation

We start with a dataset (blue) and want an algorithm that works on independent validation data (yellow).



K-fold cross validation

But we never see the independent data.



K-fold cross validation

So we split off a **training set** (blue) and a **test set** (red). Train only on the training set; use the test set only for evaluation.



K-fold cross validation

We need to tune parameters (λ) without touching the test set, and tuning and evaluating on the same data overtrains.

Cross validation estimates the MSE for each candidate λ; we pick the smallest.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

K-fold cross validation

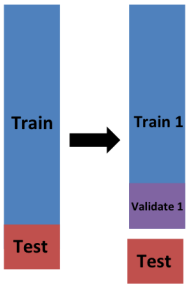
For a fixed λ, we want to imitate

$$\text{MSE}(\lambda) = \frac{1}{B}\sum_{b=1}^{B}\frac{1}{N}\sum_{i=1}^{N}\left(\hat{y}_i^b(\lambda) - y_i^b\right)^2$$

K-fold cross validation creates K pseudo-independent samples. Here $K = 5$.

K-fold cross validation

Pick $M = N/K$ observations at random as the *validation set* $y_1^b, \ldots, y_M^b$ (here $b = 1$):



WEJ LAB
RUTGERS HEALTH
Center for Data Science

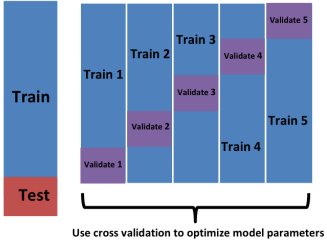
K-fold cross validation

Fit on the rest of the training set, then compute the error on the validation set:

$$\hat{\mathrm{MSE}}_b(\lambda) = \frac{1}{M} \sum_{i=1}^{M} \left(\hat{y}_i^b(\lambda) - y_i^b \right)^2$$

K-fold cross validation

Split the observations into K non-overlapping folds:



K-fold cross validation

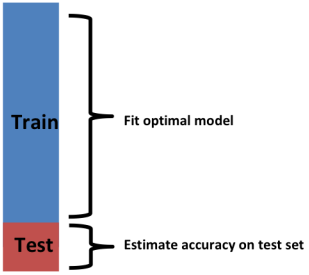
Repeat for each fold $b = 1, \ldots, K$ and average:

$$\hat{\text{MSE}}(\lambda) = \frac{1}{K} \sum_{b=1}^{K} \hat{\text{MSE}}_b(\lambda)$$

Then pick the λ that minimizes $\hat{\text{MSE}}(\lambda)$.

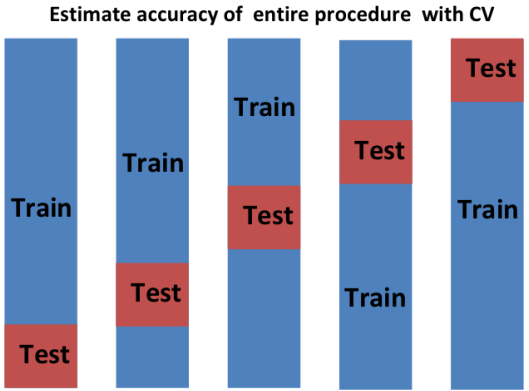
K-fold cross validation

Tuning used the training data, so evaluate the final model on the test set:

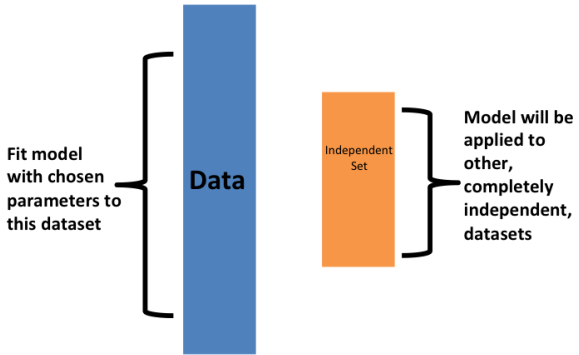


K-fold cross validation

Or cross validate the whole procedure again:



K-fold cross validation



K-fold cross validation

Choosing K:

- Larger K: training folds look more like the full data
- But slower: 100-fold is 10x slower than 10-fold
- $K = 5$ and $K = 10$ are popular choices

WEJ LAB
RUTGERS HEALTH
Center for Data Science

K-fold cross validation

To reduce variance, we can take more samples: pick sets of observations at random instead of fixed folds.

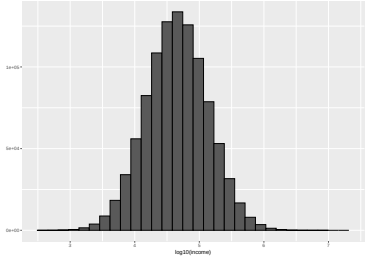
Sampling **with replacement** is the *bootstrap*, and it is the default in **caret**. Here is how the bootstrap works in general.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Bootstrap

Suppose the income distribution of your population is as follows:

```
set.seed(1995)
n <- 10^6
income <- 10^(rnorm(n, log10(45000), log10(3)))
qplot(log10(income), bins = 30, color = I("black"))
```



Bootstrap

The population median is:

```
m <- median(income)
m
```

```
## [1] 44938.54
```

Bootstrap

In practice we only have a sample. Estimate m with the median M of a sample of 100:

```
N <- 100
X <- sample(income, N)
median(X)
```

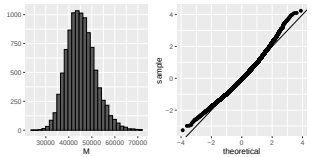
```
## [1] 38461.33
```

How to construct a CI? What is the distribution of M?

Bootstrap

We can use a Monte Carlo simulation to learn the distribution of M.

```
library(gridExtra)
B <- 10^4
M <- replicate(B, {X <- sample(income, N); median(X)})
p1 <- qplot(M, bins = 30, color = I("black"))
p2 <- qplot(sample = scale(M), xlab = "theoretical",
            ylab = "sample") + geom_abline()
grid.arrange(p1, p2, ncol = 2)
```



Bootstrap

In practice we don't know this distribution. A 95% CI based on the CLT:

```
median(X) + 1.96 * sd(X) / sqrt(N) * c(-1, 1)
```

```
## [1] 21017.93 55904.72
```

Bootstrap

is quite different from the CI from the actual distribution of M:

```
quantile(M, c(0.025, 0.975))
```

```
##      2.5%    97.5%
## 34437.72 59049.59
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Bootstrap

Bootstrap: resample X with replacement to approximate the distribution of M:

```
B <- 10^4
M_star <- replicate(B, {
  X_star <- sample(X, N, replace = TRUE)
  median(X_star)
})
```

The bootstrap CI is much closer to the true one:

```
quantile(M_star, c(0.025, 0.975))
```

```
##     2.5%    97.5%
## 30252.82 56908.62
```

Bootstrap

For more, including bias corrections for bootstrap CIs, see Efron & Tibshirani, *An Introduction to the Bootstrap*.

In cross validation, we can bootstrap many times instead of dividing the data into equal folds.

The caret package

Every ML package in R has its own syntax. **caret** gives one consistent interface to 237 methods[1].

caret does not install the underlying packages; the manual lists what each method needs.

[1] https://topepo.github.io/caret/available-models.html

The caret `train` function

caret can also run cross validation for us. We use the 2 or 7 example again:

```
library(tidyverse)
library(dslabs)
data("mnist_27")
```

The caret `train` function

`train` fits different algorithms with the same syntax:

```
library(caret)
train_glm <- train(y ~ ., method = "glm",
                   data = mnist_27$train)
train_knn <- train(y ~ ., method = "knn",
                   data = mnist_27$train)
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

The caret `train` function

Prediction code is also the same for every method (`predict.train`):

```
y_hat_glm <- predict(train_glm,
                     mnist_27$test, type = "raw")
y_hat_knn <- predict(train_knn,
                     mnist_27$test, type = "raw")
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

The caret `train` function

So comparing algorithms is quick:

```
confusionMatrix(y_hat_glm,
      mnist_27$test$y)$overall[["Accuracy"]]
```

```
## [1] 0.75
```

```
confusionMatrix(y_hat_knn,
      mnist_27$test$y)$overall[["Accuracy"]]
```

```
## [1] 0.84
```

Cross validation

For algorithms with tuning parameters, `train` automatically cross validates over a few default values. To see which parameters are tuned:

```
getModelInfo("knn")
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Cross validation

or the quicker:

```
modelLookup("knn")
```

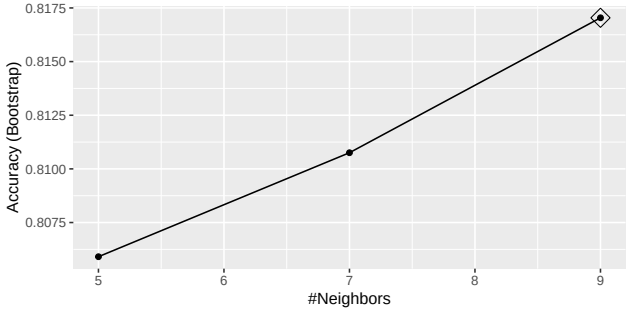
Run with defaults:

```
train_knn <- train(y ~ ., method = "knn",
                   data = mnist_27$train)
```

Cross validation

Plot the CV results; `highlight = TRUE` marks the best:

```
ggplot(train_knn, highlight = TRUE)
```



Cross validation

Defaults: 25 bootstrap samples, and for kNN only $k = 5, 7, 9$.

Change the candidates with `tuneGrid`: a data frame whose column names match `modelLookup`.

Cross validation

Try 32 values of k from 9 to 71: `data.frame(k = seq(9, 71, 2))`.

That is $32 \times 25 = 800$ kNN fits.

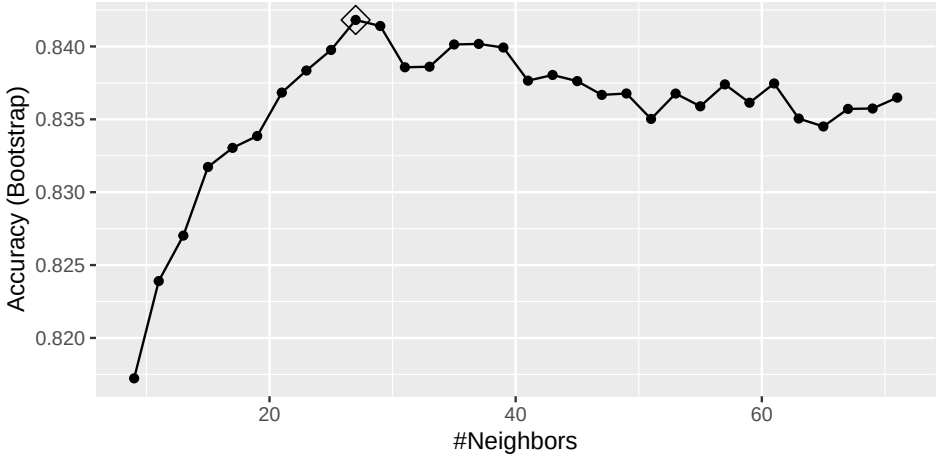
WEJ LAB
RUTGERS HEALTH
Center for Data Science

Cross validation

Cross validation is random, so set the seed for reproducibility:

```
set.seed(2008)
train_knn <- train(y ~ ., method = "knn",
      data = mnist_27$train,
      tuneGrid = data.frame(k = seq(9, 71, 2)))
ggplot(train_knn, highlight = TRUE)
```

Cross validation



Cross validation

The best parameter:

```
train_knn$bestTune
```

```
##     k
## 10 27
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Cross validation

and the best model:

```
train_knn$finalModel
```

```
## 27-nearest neighbor model
## Training set outcome distribution:
##
##   2   7
## 379 421
```

Cross validation

`predict` uses the best model. Test-set accuracy (the test set was not used in cross validation):

```
confusionMatrix(predict(train_knn,
                        mnist_27$test, type = "raw"),
                mnist_27$test$y)$overall["Accuracy"]
```

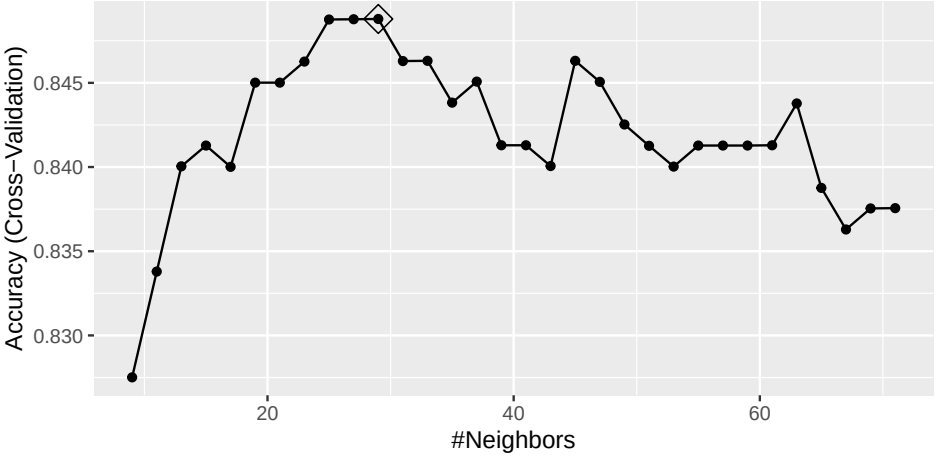
```
## Accuracy
##    0.835
```

Cross validation

`trainControl` changes the CV method; 10-fold is faster than 25 bootstraps:

```
control <- trainControl(method = "cv",
                        number = 10, p = .9)
train_knn_cv <- train(y ~ ., method = "knn",
                      data = mnist_27$train,
                      tuneGrid = data.frame(k = seq(9, 71, 2)),
                      trControl = control)
ggplot(train_knn_cv, highlight = TRUE)
```

Cross validation

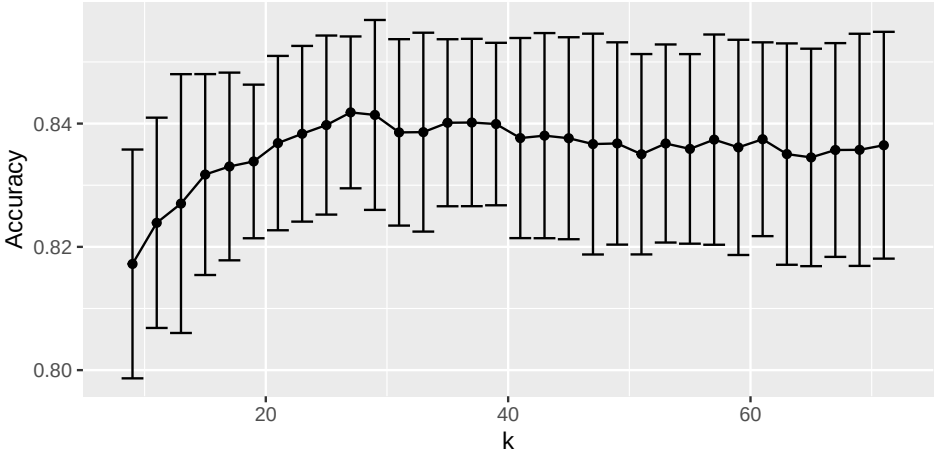


Cross validation

Accuracy with standard deviation bars from the CV samples:

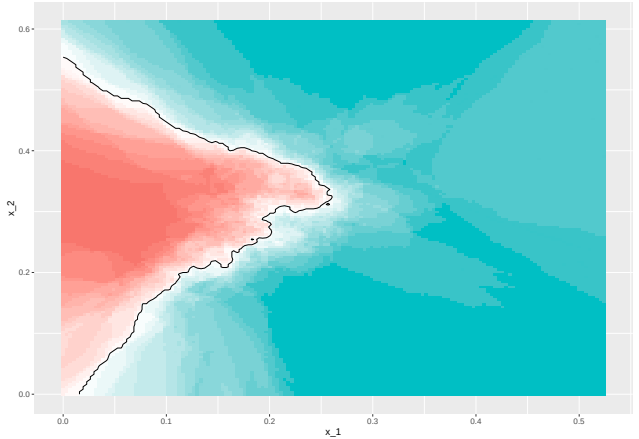
```
train_knn$results %>%
  ggplot(aes(x = k, y = Accuracy)) +
  geom_line() + geom_point() +
  geom_errorbar(aes(x = k,
                    ymin = Accuracy - AccuracySD,
                    ymax = Accuracy + AccuracySD))
```

Cross validation



Example: fitting with loess

The best kNN model approximates the true conditional probability:



Example: fitting with loess

But the boundary is wiggly: kNN, like a bin smoother, uses no kernel. Loess is smoother; in **caret** it is the `gamLoess` method[2], which needs the **gam** package:

```
install.packages("gam")
```

[2] https://topepo.github.io/caret/available-models.html

Example: fitting with loess

It has two tuning parameters:

```
modelLookup("gamLoess")
```

```
##      model parameter  label forReg forClass probModel
## 1 gamLoess      span   Span   TRUE     TRUE      TRUE
## 2 gamLoess    degree Degree   TRUE     TRUE      TRUE
```

We tune `span` and fix `degree = 1` (the grid still needs both columns):

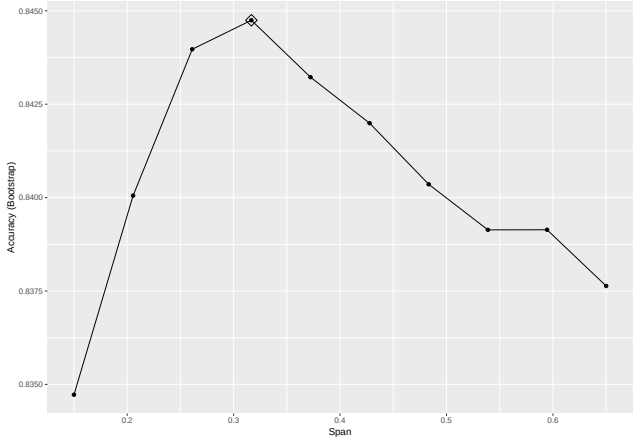
```
grid <- expand.grid(span = seq(0.15, 0.65, len = 10)
                    , degree = 1)
```

Example: fitting with loess

Using the default CV settings:

```
train_loess <- train(y ~ ., method = "gamLoess",
                     tuneGrid=grid,
                     data = mnist_27$train)
ggplot(train_loess, highlight = TRUE)
```

Example: fitting with loess



Example: fitting with loess

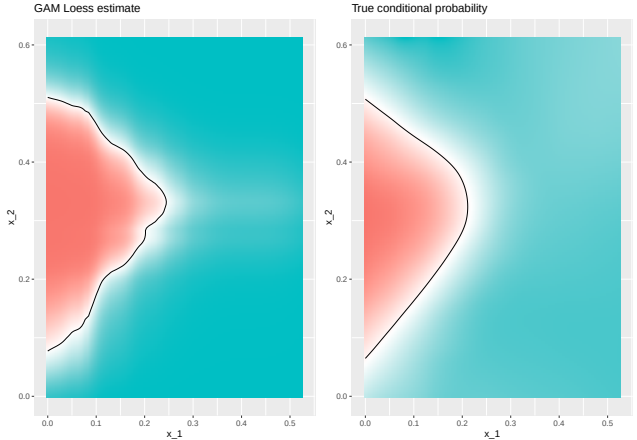
Accuracy is similar to kNN:

```
confusionMatrix(data = predict(
      train_loess, mnist_27$test),
      reference = mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##     0.85
```

Example: fitting with loess

but the estimate is smoother:



Application: developing a TB biomarker

Back to the TB NanoString data: 179 people (active TB vs. latent infection), 107 genes.

- p (107 genes) is close to N (179 people): easy to overfit
- Goal: a classifier that works on **new** patients
- Cross-validation is how we choose and honestly evaluate it

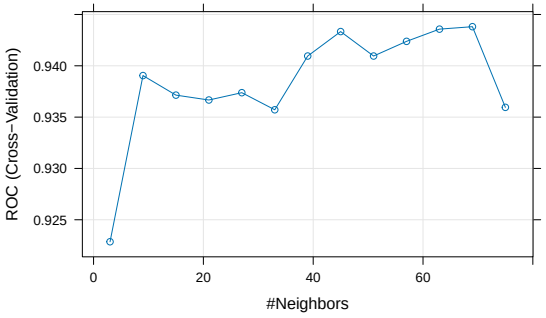
```
tb <- readRDS("TBnanostring.rds")
tb$TB_Status <- factor(tb$TB_Status, levels = c("LTBI", "TB"))
```

kNN on all 107 genes, tuned by CV

```
set.seed(1)
idx <- createDataPartition(tb$TB_Status, p = 0.7, list = FALSE)
tb_train <- tb[idx, ]; tb_test <- tb[-idx, ]
ctrl <- trainControl(method = "cv", number = 10, classProbs = TRUE,
                     summaryFunction = twoClassSummary)
fit_knn <- train(TB_Status ~ ., data = tb_train, method = "knn",
                 preProcess = c("center", "scale"), metric = "ROC",
                 tuneGrid = data.frame(k = seq(3, 75, 6)), trControl = ctrl)
fit_knn$bestTune
```

```
##     k
## 12 69
```

kNN on all 107 genes, tuned by CV



Cross-validated AUC for each k; `train` picks the best and refits on all the training data.

Checking on the test set

```
p_knn <- predict(fit_knn, tb_test, type = "prob")[, "TB"]
pROC::auc(pROC::roc(tb_test$TB_Status, p_knn, levels = c("LTBI", "TB"),
                    direction = "<", quiet = TRUE))
```

```
## Area under the curve: 0.9275
```

```
confusionMatrix(predict(fit_knn, tb_test), tb_test$TB_Status,
                positive = "TB")$byClass[c("Sensitivity", "Specificity")]
```

```
## Sensitivity Specificity
##   0.7391304   0.9666667
```

The CV estimate (0.94) and the test AUC are close: CV gave a fair estimate without touching the test set.

A common pitfall: selecting genes before CV

A tempting shortcut:

1. Use **all** samples to find the 10 genes that best separate TB from LTBI
2. Then cross-validate a classifier built on those 10 genes

The CV error is now **optimistic**: the test folds already helped choose the genes. This **selection bias** has inflated many published biomarkers (Ambroise & McLachlan, *PNAS*, 2002).

Demonstrating selection bias

Shuffle the labels so **no** gene is truly related to TB status, then compare selecting genes outside vs. inside the CV loop:

```
X <- scale(as.matrix(tb[, -1]))
score_cv <- function(X, y, inside, ntop = 10) {
  pick <- function(Xs, ys) {                  # top genes by t-statistic
    t <- apply(Xs, 2, function(g) t.test(g[ys == "TB"], g[ys == "LTBI"])$statistic)
    top <- order(abs(t), decreasing = TRUE)[1:ntop]
    list(top = top, sign = sign(t[top])) }
  folds <- createFolds(y, k = 5); p <- numeric(length(y))
  if (!inside) sel <- pick(X, y)              # WRONG: uses all samples
  for (f in folds) {
    if (inside) sel <- pick(X[-f, ], y[-f])   # RIGHT: training folds only
    p[f] <- X[f, sel$top, drop = FALSE] %*% sel$sign }
  as.numeric(pROC::auc(pROC::roc(y, p, levels = c("LTBI", "TB"),
                                 direction = "<", quiet = TRUE))) }
set.seed(1)
res <- replicate(20, { y_shuffled <- sample(tb$TB_Status)
  c(outside = score_cv(X, y_shuffled, FALSE), inside = score_cv(X, y_shuffled, TRUE)) })
round(rowMeans(res), 2)
```

Demonstrating selection bias

With labels that carry **no** information:

- Genes selected **outside** CV: AUC ≈ 0.64, a "biomarker" out of pure noise
- Genes selected **inside** CV: AUC ≈ 0.5, correctly a coin flip
- With the real labels (inside CV): AUC ≈ 0.93

Biomarker development: lessons

- Every data-driven step (gene selection, scaling, tuning) goes **inside** the CV loop (caret: `preProcess`, `sbf`, `rfe`)
- Keep a test set untouched until the model is **locked**
- Then validate in an **independent cohort** (different site, time, or population)
- Small studies give noisy estimates: report confidence intervals

Summary

- Training error is optimistic; we need an honest estimate of test error
- Small k in kNN over-trains; large k over-smooths
- K-fold CV and the bootstrap estimate test error from the training data
- Tune parameters by CV; keep the test set for the final check
- caret's `train` handles resampling and tuning for hundreds of models
- Biomarkers: put gene selection inside CV, lock the model, then validate in an independent cohort

Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/Denver
## tzcode source: system (glibc)
##
## attached base packages:
## [1] splines   stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] gridExtra_2.3   dslabs_0.7.6    gam_1.22-3      foreach_1.5.2
##  [5] lubridate_1.9.3 forcats_1.0.0   stringr_1.5.1   dplyr_1.1.4
##  [9] purrr_1.0.2     readr_2.1.5     tidyr_1.3.1     tibble_3.2.1
## [13] tidyverse_2.0.0 caret_6.0-94    lattice_0.22-5  ggplot2_3.4.4
##
## loaded via a namespace (and not attached):
## [1] gtable_0.3.4        xfun_0.41           recipes_1.0.9
```